Product Requirement Document: PulseFlow (MVP) - Updated

1. Project Overview

PulseFlow (formerly ClipsToThreads) is a B2C/B2B Micro-SaaS platform designed for content creators, digital marketers, and crypto-influencers. The platform automates the process of content repurposing by taking a YouTube video URL (or audio file), transcribing the content, and using LLMs to transform the transcript into highly engaging, platform-specific text formats (Twitter/X Threads, LinkedIn long-form posts, and Telegram articles).

2. Core Tech Stack

- **Frontend:** Next.js (App Router), Tailwind CSS, TypeScript, Lucide React (Icons), shadcn/ui (Component base).
- **Backend & Database:** Supabase (PostgreSQL, Supabase Auth, Row Level Security).
- **AI Integration:** OpenAI API (Whisper for audio transcription) or dedicated YouTube Transcript API + Anthropic Claude 3.5 Sonnet API (via Vercel AI SDK or direct REST) for content generation.
- **Infrastructure:** Vercel (Hosting & Edge Functions).
- **Monetization:** Crypto Payment Gateway (e.g., Cryptomus/CryptoBot) or Stripe (integrated via webhooks).

3. Database Schema (Supabase PostgreSQL)

3.1. `users` table

- **Purpose:** Extends Supabase `auth.users` table with application-specific user profiles and credit management.
- **Fields:**
  - `id` (uuid, FK to `auth.users.id`): Primary key, foreign key linking to Supabase's authentication user table.
  - `credits` (integer, default 3): Represents the number of content generation credits available to the user. Default value for free tier.

    - `subscription_status` (text/enum, default 'free'): Indicates the user's subscription level (e.g., 'free', 'pro', 'premium').

- **Row Level Security (RLS) Considerations:**
    - Users should only be able to view and update their own `credits` and `subscription_status`.
    - `auth.uid()` function will be crucial for implementing RLS policies.

3.2. `generations` table

- **Purpose:** Tracks user activity and stores the results of content generation.
- **Fields:**
    - `id` (uuid, primary key, default `gen_random_uuid()`): Unique identifier for each generation record.
    - `user_id` (uuid, FK to `public.users.id`): Foreign key linking to the `users` table, indicating which user initiated the generation.
    - `video_url` (text): The YouTube video URL provided by the user.
    - `raw_transcript` (text): The full transcript extracted from the YouTube video.
    - `output_x` (text): The generated content formatted for Twitter/X Threads.
    - `output_linkedin` (text): The generated content formatted for LinkedIn long-form posts.
    - `output_telegram` (text): The generated content formatted for Telegram articles.
    - `created_at` (timestamp with time zone, default `now()`): Timestamp of when the generation record was created.

- **Row Level Security (RLS) Considerations:**
    - Users should only be able to view their own `generations` records.
    - `auth.uid()` will be used to filter records based on the authenticated user.

4. Development Plan (Phase 1 to Phase N)

This section outlines the step-by-step development plan for the PulseFlow MVP, providing explicit requirements, context, and code logic instructions for each phase. This plan is designed to be directly actionable within a development environment like Cursor IDE.

Phase 1: Project Setup & Basic Authentication

Objective: Initialize the Next.js project, configure Supabase, and implement basic user authentication (Google OAuth and Magic Link) with extended user profiles, utilizing the updated `@supabase/ssr` package.

4.1.1. Project Initialization

- **Requirements:** Create a new Next.js project with TypeScript, Tailwind CSS, and shadcn/ui.
- **Context:** Establishes the foundational frontend and styling framework.
- **Code Logic Instructions:**
  1. **Create Next.js App:**

```
npx create-next-app@latest pulseflow --typescript --tailwind --esli
cd pulseflow
```

  2. **Initialize shadcn/ui:**

```
npx shadcn-ui@latest init
# Follow prompts: select 'New York' style, 'Slate' color, CSS varia
# Configure `components.json` to use `src/components/ui`.
```

  3. **Install Supabase Client Libraries (Updated):**

```
npm uninstall @supabase/auth-helpers-nextjs # If previously install
npm install @supabase/ssr @supabase/supabase-js
```

  4. **Environment Variables:** Create a `.env.local` file in the project root:

```
NEXT_PUBLIC_SUPABASE_URL=YOUR_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY=YOUR_SUPABASE_ANON_KEY
```

Replace placeholders with actual Supabase project URL and Anon Key from your Supabase project settings.

4.1.2. Supabase Project Setup & Database Schema

- **Requirements:** Create a Supabase project, define `users` and `generations` tables, and configure Row Level Security (RLS).
- **Context:** Sets up the backend database and ensures secure data access.
- **Code Logic Instructions:**
  1. **Create Supabase Project:** Go to `database.new` and create a new project.
  2. **SQL Editor - `users` table:** Execute the following SQL to create the `users` table and RLS policies.

```sql
-- Create the public.users table
CREATE TABLE public.users (
    id uuid REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY
    credits integer DEFAULT 3 NOT NULL,
    subscription_status text DEFAULT 'free' NOT NULL
);

-- Enable RLS on public.users
ALTER TABLE public.users ENABLE ROW LEVEL SECURITY;

-- Policy for users to view their own profile
CREATE POLICY "Users can view their own profile" ON public.users
FOR SELECT USING (auth.uid() = id);

-- Policy for users to update their own profile (e.g., subscription
CREATE POLICY "Users can update their own profile" ON public.users
FOR UPDATE USING (auth.uid() = id);

-- Function to create a public.users entry on new auth.users signup
CREATE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.users (id, credits, subscription_status)
    VALUES (NEW.id, 3, 'free');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Trigger to call handle_new_user on auth.users insert
CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

3. **SQL Editor - `generations` table:** Execute the following SQL to create the `generations` table and RLS policies.

```
-- Create the public.generations table
CREATE TABLE public.generations (
    id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id uuid REFERENCES public.users(id) ON DELETE CASCADE NOT
    video_url text NOT NULL,
    raw_transcript text,
    output_x text,
    output_linkedin text,
    output_telegram text,
    created_at timestamp with time zone DEFAULT now() NOT NULL
);

-- Enable RLS on public.generations
ALTER TABLE public.generations ENABLE ROW LEVEL SECURITY;

-- Policy for users to view their own generations
CREATE POLICY "Users can view their own generations" ON public.gene
FOR SELECT USING (auth.uid() = user_id);

-- Policy for users to insert their own generations
CREATE POLICY "Users can insert their own generations" ON public.ge
FOR INSERT WITH CHECK (auth.uid() = user_id);
```

4.1.3. Supabase Client Initialization (Updated)

- **Requirements:** Create Supabase client instances for both client-side and server-side operations using `@supabase/ssr`.
- **Context:** Provides a centralized and correct way to interact with Supabase services across the Next.js App Router.
- **Code Logic Instructions:**
  1. Create `src/lib/supabase/client.ts` for client components:

```
import { createBrowserClient } from '@supabase/ssr';

export function createClient() {
    return createBrowserClient(
        process.env.NEXT_PUBLIC_SUPABASE_URL!,
        process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
    );
}
```

2. Create `src/lib/supabase/server.ts` for server components and Route Handlers:

```
import { createServerClient, type CookieOptions } from '@supabase/s
import { cookies } from 'next/headers';

export function createServerSupabaseClient() {
    const cookieStore = cookies();

    return createServerClient(
        process.env.NEXT_PUBLIC_SUPABASE_URL!,
        process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
        {
            cookies: {
                get(name: string) {
                    return cookieStore.get(name)?.value;
                },
                set(name: string, value: string, options: CookieOpt
                    cookieStore.set({ name, value, ...options });
                },
                remove(name: string, options: CookieOptions) {
                    cookieStore.set({ name, value: '', ...options }
                },
            },
        }
    );
}
```

4.1.4. Authentication UI & Logic (Updated)

- **Requirements:** Implement Google OAuth and Magic Link sign-in/sign-up flows using the new `@supabase/ssr` client.
- **Context:** Enables users to access the platform securely.
- **Code Logic Instructions:**
    1. **Supabase Auth Provider Setup:** In your Supabase project settings, enable Google as an Auth Provider and configure the redirect URLs (e.g., `http://localhost:3000/auth/callback` and your deployed domain).
    2. **Auth Callback Route (`src/app/auth/callback/route.ts`):** Handle Supabase OAuth redirects.

```
import { createServerSupabaseClient } from '@/lib/supabase/server';
import { NextResponse } from 'next/server';

export async function GET(request: Request) {
    const requestUrl = new URL(request.url);
    const code = requestUrl.searchParams.get('code');

    if (code) {
        const supabase = createServerSupabaseClient();
        await supabase.auth.exchangeCodeForSession(code);
    }

    // URL to redirect to after sign in process completes
    return NextResponse.redirect(requestUrl.origin);
}
```

    3. **Login Component (`src/components/auth/AuthForm.tsx`):** Create a component for login/signup.

```tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createClient } from '@/lib/supabase/client'; // Updated im
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export default function AuthForm() {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const router = useRouter();
    const supabase = createClient(); // Updated client initializati

    const handleSignIn = async () => {
        await supabase.auth.signInWithPassword({ email, password })
        router.refresh();
    };

    const handleSignUp = async () => {
        await supabase.auth.signUp({ email, password, options: { em
        router.refresh();
    };

    const handleGoogleSignIn = async () => {
        await supabase.auth.signInWithOAuth({
            provider: 'google',
            options: {
                redirectTo: `${location.origin}/auth/callback`,
            },
        });
    };

    return (
        <div className="flex flex-col space-y-4">
            <div>
                <Label htmlFor="email">Email</Label>
                <Input id="email" type="email" value={email} onChan
            </div>
            <div>
                <Label htmlFor="password">Password</Label>
```

```
                <Input id="password" type="password" value={passwor
            </div>
            <Button onClick={handleSignIn}>Sign In</Button>
            <Button onClick={handleSignUp}>Sign Up</Button>
            <Button onClick={handleGoogleSignIn}>Sign In with Googl
        </div>
    );
}
```

4. **Integrate AuthForm into a page (e.g., `src/app/login/page.tsx`):**

```
import AuthForm from '@/components/auth/AuthForm';

export default function LoginPage() {
    return (
        <div className="flex min-h-screen items-center justify-cent
            <div className="w-full max-w-md rounded-lg bg-gray-800
                <h1 className="mb-6 text-center text-2xl font-bold
                <AuthForm />
            </div>
        </div>
    );
}
```

4.1.5. User Session Management (Updated)

- **Requirements:** Display user session status and provide a logout mechanism using the new `@supabase/ssr` client.
- **Context:** Essential for a functional authenticated application.
- **Code Logic Instructions:**
    1. **User Info Component (e.g., `src/components/layout/UserNav.tsx`):**

```tsx
import { createServerSupabaseClient } from '@/lib/supabase/server';
import { Button } from '@/components/ui/button';
import { redirect } from 'next/navigation';

export default async function UserNav() {
    const supabase = createServerSupabaseClient();
    const { data: { user } } = await supabase.auth.getUser();

    const signOut = async () => {
        'use server';
        const supabase = createServerSupabaseClient();
        await supabase.auth.signOut();
        redirect('/login');
    };

    return user ? (
        <div className="flex items-center space-x-4">
            <span className="text-white">Hello, {user.email}</span>
            <form action={signOut}>
                <Button type="submit">Sign Out</Button>
            </form>
        </div>
    ) : (
        <Button onClick={() => redirect('/login')}>Sign In</Button>
    );
}
```

2. **Integrate UserNav into Layout (e.g., `src/app/layout.tsx` or `src/app/dashboard/layout.tsx`):**

```
// Example integration in a layout file
import UserNav from '@/components/layout/UserNav';

export default function RootLayout({ children }: { children: React.
    return (
        <html lang="en">
            <body>
                <header className="bg-gray-800 p-4 flex justify-end
                    <UserNav />
                </header>
                <main>{children}</main>
            </body>
        </html>
    );
}
```

4.1.6. Acceptance Criteria for Phase 1:

- Next.js application successfully initializes with Tailwind CSS and shadcn/ui.
- Supabase project is created, and `users` and `generations` tables are correctly defined with RLS.
- New users signing up via Google OAuth or Magic Link are automatically populated in the `public.users` table with default credits.
- Existing users can sign in and their session is maintained.
- Users can sign out, and they are redirected to the login page.
- Basic UI elements for authentication are functional and responsive.
- All Supabase client interactions utilize the `@supabase/ssr` package, ensuring compatibility and best practices for Next.js App Router.

Phase 2: Dashboard UI & YouTube Transcript Extraction

Objective: Develop the main dashboard UI, including the YouTube URL input field and a “Generate” button. Implement the YouTube transcript extraction logic.

4.2.1. Dashboard UI Development

- **Requirements:** Create a premium dark-themed minimalist dashboard with a YouTube URL input and a “Generate” button.

- **Context:** This is the primary user interface for interacting with the PulseFlow platform.
- **Code Logic Instructions:**
    1. **Dashboard Page (`src/app/dashboard/page.tsx`):**

```
'use client';

import { useState } from 'react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from '@/compone
import { Label } from '@/components/ui/label';

export default function DashboardPage() {
    const [youtubeUrl, setYoutubeUrl] = useState('');
    const [isLoading, setIsLoading] = useState(false);

    const handleGenerate = async () => {
        setIsLoading(true);
        // TODO: Implement generation logic
        console.log('Generating content for:', youtubeUrl);
        setIsLoading(false);
    };

    return (
        <div className="min-h-screen bg-gray-900 text-white p-8">
            <div className="max-w-3xl mx-auto">
                <h1 className="text-4xl font-bold mb-8 text-center"
                <Card className="bg-gray-800 border-gray-700">
                    <CardHeader>
                        <CardTitle className="text-white">Generate
                    </CardHeader>
                    <CardContent>
                        <div className="space-y-4">
                            <div>
                                <Label htmlFor="youtube-url" classN
                                <Input
                                    id="youtube-url"
                                    type="url"
                                    placeholder="e.g., https://www.
                                    value={youtubeUrl}
                                    onChange={(e) => setYoutubeUrl(
                                    className="bg-gray-700 border-g
                                />
                            </div>
                            <Button
                                onClick={handleGenerate}
```

```
                                disabled={!youtubeUrl || isLoading}
                                className="w-full bg-blue-600 hover
                            >
                                {isLoading ? 'Generating...' : 'Gen
                            </Button>
                        </div>
                    </CardContent>
                </Card>
            </div>
        </div>
    );
}
```

2. **Ensure Dark Theme:** Verify `tailwind.config.js` is configured for dark mode, or add `dark` class to `html` tag in `src/app/layout.tsx`.

4.2.2. YouTube Transcript Extraction API

- **Requirements:** Create an API endpoint to fetch transcripts from a given YouTube URL.
- **Context:** This endpoint will be called from the frontend to get the raw video content.
- **Code Logic Instructions:**
  1. **Install `youtube-transcript-api` (or similar library):**

```
npm install youtube-transcript-api
# Or consider a serverless approach using a dedicated service if di
```

  2. **API Route (`src/app/api/transcript/route.ts`):**

```
import { NextResponse } from 'next/server';
import { YoutubeTranscript } from 'youtube-transcript';

export async function POST(request: Request) {
    try {
        const { videoUrl } = await request.json();

        if (!videoUrl) {
            return NextResponse.json({ error: 'Video URL is require
        }

        const videoIdMatch = videoUrl.match(/(?:youtu\.be\/|youtube
        if (!videoIdMatch || !videoIdMatch[1]) {
            return NextResponse.json({ error: 'Invalid YouTube URL'
        }
        const videoId = videoIdMatch[1];

        const transcript = await YoutubeTranscript.fetchTranscript(
        const rawTranscript = transcript.map(item => item.text).joi

        return NextResponse.json({ rawTranscript });
    } catch (error) {
        console.error('Error fetching transcript:', error);
        return NextResponse.json({ error: 'Failed to fetch transcri
    }
}
```

4.2.3. Integrate Transcript Extraction into Dashboard

- **Requirements:** Call the transcript API from the dashboard and store the raw transcript.
- **Context:** Connects the UI with the backend transcript service.
- **Code Logic Instructions:**
    1. **Update `handleGenerate` in `src/app/dashboard/page.tsx`:**

```
// ... inside DashboardPage component
const handleGenerate = async () => {
    setIsLoading(true);
    try {
        // 1. Deduct Credit (placeholder for now, actual logic in P

        // 2. Fetch Transcript
        const response = await fetch('/api/transcript', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ videoUrl: youtubeUrl }),
        });

        if (!response.ok) {
            const errorData = await response.json();
            throw new Error(errorData.error || 'Failed to fetch tra
        }

        const data = await response.json();
        const rawTranscript = data.rawTranscript;
        console.log('Raw Transcript:', rawTranscript);
        // TODO: Store rawTranscript in state or pass to next step

    } catch (error) {
        console.error('Generation error:', error);
        alert(`Error: ${error.message}`);
    } finally {
        setIsLoading(false);
    }
};
// ... rest of the component
```

4.2.4. Acceptance Criteria for Phase 2:

- Dashboard UI is rendered with a dark theme, YouTube URL input, and a “Generate” button.
- Entering a valid YouTube URL and clicking “Generate” successfully fetches the transcript via the API route.
- Invalid YouTube URLs or errors during transcript fetching are handled gracefully and displayed to the user.

- The raw transcript is logged to the console (placeholder for further processing).

Phase 3: Credit Check & Atomic Credit Deduction

Objective: Implement the credit check mechanism before content generation and ensure atomic deduction of user credits using Supabase functions.

4.3.1. Server-side Credit Check & Deduction Function

- **Requirements:** Create a Supabase PostgreSQL function to atomically check user credits and deduct one if available.
- **Context:** Prevents race conditions and ensures data integrity for credit management.
- **Code Logic Instructions:**
    1. **SQL Editor - `deduct_credit` function:** Execute the following SQL in Supabase SQL Editor.

```
CREATE OR REPLACE FUNCTION public.deduct_credit(p_user_id uuid)
RETURNS TABLE(new_credits integer, success boolean, message text)
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    current_credits integer;
BEGIN
    -- Ensure only the user themselves can call this function for t
    IF p_user_id IS DISTINCT FROM auth.uid() THEN
        RETURN QUERY SELECT -1, FALSE, 'Unauthorized: You can only
        RETURN;
    END IF;

    -- Lock the row for the current user to prevent race conditions
    SELECT credits INTO current_credits FROM public.users WHERE id

    IF current_credits IS NULL THEN
        RETURN QUERY SELECT -1, FALSE, 'User not found.';
    ELSIF current_credits > 0 THEN
        UPDATE public.users
        SET credits = credits - 1
        WHERE id = p_user_id
        RETURNING credits INTO new_credits;
        RETURN QUERY SELECT new_credits, TRUE, 'Credit deducted suc
    ELSE
        RETURN QUERY SELECT 0, FALSE, 'Insufficient credits.';
    END IF;
END;
$$;

-- Grant usage to authenticated role
GRANT EXECUTE ON FUNCTION public.deduct_credit(uuid) TO authenticat
```

4.3.2. API Route for Credit Management (Updated)

- **Requirements:** Create an API endpoint to call the `deduct_credit` Supabase function using the new `@supabase/ssr` server client.

- **Context:** Provides a secure and controlled way for the frontend to interact with the credit system.
- **Code Logic Instructions:**
    1. **API Route (`src/app/api/deduct-credit/route.ts`):**

```
import { NextResponse } from 'next/server';
import { createServerSupabaseClient } from '@/lib/supabase/server';

export async function POST(request: Request) {
    const supabase = createServerSupabaseClient(); // Updated clien
    const { data: { user } } = await supabase.auth.getUser();

    if (!user) {
        return NextResponse.json({ error: 'Unauthorized' }, { statu
    }

    try {
        const { data, error } = await supabase.rpc('deduct_credit',

        if (error) {
            console.error('Supabase RPC error:', error);
            return NextResponse.json({ error: error.message }, { st
        }

        const result = data[0]; // Assuming data is an array with o

        if (!result.success) {
            return NextResponse.json({ error: result.message }, { s
        }

        return NextResponse.json({ newCredits: result.new_credits,
    } catch (error) {
        console.error('Error deducting credit:', error);
        return NextResponse.json({ error: 'Internal server error' }
    }
}
```

4.3.3. Integrate Credit Check & Deduction into Dashboard

- **Requirements:** Modify the `handleGenerate` function to perform a credit check and deduction before proceeding with AI generation.
- **Context:** Enforces the monetization model.
- **Code Logic Instructions:**
    1. **Update `handleGenerate` in `src/app/dashboard/page.tsx`:**

```
// ... inside DashboardPage component
import { useRouter } from 'next/navigation';

export default function DashboardPage() {
    // ... existing states
    const router = useRouter();

    const handleGenerate = async () => {
        setIsLoading(true);
        try {
            // 1. Deduct Credit
            const deductResponse = await fetch('/api/deduct-credit'
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
            });

            if (!deductResponse.ok) {
                const errorData = await deductResponse.json();
                if (deductResponse.status === 402) {
                    alert('Insufficient credits. Redirecting to pri
                    router.push('/pricing'); // Redirect to pricing
                } else {
                    throw new Error(errorData.error || 'Failed to d
                }
                return; // Stop execution if credit deduction fails
            }
            const { newCredits } = await deductResponse.json();
            console.log('Credits remaining:', newCredits);
            // TODO: Update UI to reflect newCredits

            // 2. Fetch Transcript (existing logic)
            const transcriptResponse = await fetch('/api/transcript
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ videoUrl: youtubeUrl }),
            });

            if (!transcriptResponse.ok) {
                const errorData = await transcriptResponse.json();
                throw new Error(errorData.error || 'Failed to fetch
            }
```

```
            const data = await transcriptResponse.json();
            const rawTranscript = data.rawTranscript;
            console.log('Raw Transcript:', rawTranscript);
            // TODO: Proceed with AI generation using rawTranscript

        } catch (error) {
            console.error('Generation error:', error);
            alert(`Error: ${error.message}`);
        } finally {
            setIsLoading(false);
        }
    };
    // ... rest of the component
}
```

4.3.4. Acceptance Criteria for Phase 3:

- Before transcript extraction, the system checks if the user has sufficient credits.
- If credits are 0, generation is blocked, and an alert is shown (with a placeholder for redirection).
- If credits are available, one credit is atomically deducted from the user's account.
- The credit deduction is robust against race conditions due to the `FOR UPDATE` clause in the Supabase function.
- The frontend receives the updated credit count after a successful deduction.

Phase 4: AI Content Generation (Claude 3.5 Sonnet) - Updated Prompts

Objective: Integrate Anthropic Claude 3.5 Sonnet API to transform the raw transcript into platform-specific content (Twitter/X, LinkedIn, Telegram) using aggressive, catchy, and viral-ready system prompts.

4.4.1. Claude API Integration

- **Requirements:** Set up the Anthropic Claude 3.5 Sonnet API integration, preferably using Vercel AI SDK or direct REST calls.
- **Context:** This is the core AI processing step, now with enhanced prompt engineering.
- **Code Logic Instructions:**
    1. **Install Vercel AI SDK:**

```
npm install ai anthropic
```

2. **Environment Variable:** Add your Anthropic API key to `.env.local`:

```
ANTHROPIC_API_KEY=YOUR_ANTHROPIC_API_KEY
```

3. **API Route (`src/app/api/generate-content/route.ts`):**

```
import { Anthropic } from '@anthropic-ai/sdk';
import { NextResponse } from 'next/server';

const anthropic = new Anthropic({
    apiKey: process.env.ANTHROPIC_API_KEY,
});

export async function POST(request: Request) {
    try {
        const { rawTranscript, videoUrl } = await request.json(); /

        if (!rawTranscript) {
            return NextResponse.json({ error: 'Raw transcript is re
        }

        // Define platform-specific system prompts (UPDATED for vir
        const twitterPrompt = `You are a viral content strategist a
        const linkedinPrompt = `You are a LinkedIn thought leader a
        const telegramPrompt = `You are a Telegram channel guru, cr

        // Generate content for each platform in parallel
        const [outputX, outputLinkedIn, outputTelegram] = await Pro
            anthropic.messages.create({
                model: 'claude-3-5-sonnet-20240620',
                max_tokens: 2000,
                system: twitterPrompt,
                messages: [{ role: 'user', content: rawTranscript }
            }).then(response => response.content[0].text),
            anthropic.messages.create({
                model: 'claude-3-5-sonnet-20240620',
                max_tokens: 2000,
                system: linkedinPrompt,
                messages: [{ role: 'user', content: rawTranscript }
            }).then(response => response.content[0].text),
            anthropic.messages.create({
                model: 'claude-3-5-sonnet-20240620',
                max_tokens: 2000,
                system: telegramPrompt,
                messages: [{ role: 'user', content: rawTranscript }
            }).then(response => response.content[0].text),
        ]);
```

```
        // After successful generation, store in database
        const supabase = createServerSupabaseClient(); // Updated c
        const { data: { user } } = await supabase.auth.getUser();

        if (!user) {
            return NextResponse.json({ error: 'Unauthorized' }, { s
        }

        const { error: dbError } = await supabase
            .from('generations')
            .insert({
                user_id: user.id,
                video_url: videoUrl,
                raw_transcript: rawTranscript,
                output_x: outputX,
                output_linkedin: outputLinkedIn,
                output_telegram: outputTelegram,
            });

        if (dbError) {
            console.error('Error inserting generation record:', dbE
            // Consider rolling back credit deduction here if this
            return NextResponse.json({ error: 'Failed to save gener
        }

        return NextResponse.json({ outputX, outputLinkedIn, outputT
    } catch (error) {
        console.error('Error generating content:', error);
        return NextResponse.json({ error: 'Failed to generate conte
    }
}
```

4.4.2. Store Generated Content in Database

- **Requirements:** After successful generation, store the raw transcript and all generated outputs in the `generations` table.
- **Context:** Persists the results for user history and future reference.
- **Code Logic Instructions:**
  1. **Update `src/app/api/generate-content/route.ts`:** The database insertion logic is now integrated directly into this route, as shown above, ensuring the `videoUrl` is passed correctly.

2. **Update `handleGenerate` in `src/app/dashboard/page.tsx` to pass `videoUrl` to `generate-content` API:**

```
// ... inside DashboardPage component
const handleGenerate = async () => {
    setIsLoading(true);
    try {
        // ... (Credit deduction logic)

        // ... (Transcript fetching logic)

        // 3. AI Content Generation
        const generateResponse = await fetch('/api/generate-content
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ rawTranscript, videoUrl: youtube
        });

        if (!generateResponse.ok) {
            const errorData = await generateResponse.json();
            throw new Error(errorData.error || 'Failed to generate
        }

        const generatedContent = await generateResponse.json();
        console.log('Generated Content:', generatedContent);
        // TODO: Display generated content in UI

    } catch (error) {
        console.error('Generation error:', error);
        alert(`Error: ${error.message}`);
    } finally {
        setIsLoading(false);
    }
};
// ... rest of the component
```

4.4.3. Acceptance Criteria for Phase 4:

- The `generate-content` API endpoint successfully receives the raw transcript and `videoUrl`.

- Claude 3.5 Sonnet API is invoked with the **updated, aggressive, and viral-ready** platform-specific prompts for Twitter/X, LinkedIn, and Telegram.
- Generated content for all three platforms is returned.
- The raw transcript and generated outputs are successfully stored in the `generations` table, linked to the `user_id` and `video_url`.
- Errors during AI generation or database insertion are handled.

Phase 5: Output Display & Copy to Clipboard

Objective: Display the generated content in a tabbed interface and provide a "Copy to Clipboard" button for each platform.

4.5.1. Tabbed Output Display

- **Requirements:** Use shadcn/ui Tabs component to display `output_x`, `output_linkedin`, and `output_telegram`.
- **Context:** Provides a user-friendly way to view and interact with the generated content.
- **Code Logic Instructions:**
  1. **Install shadcn/ui Tabs component:**

     ```
     npx shadcn-ui@latest add tabs
     ```

  2. **Update `src/app/dashboard/page.tsx` to display generated content:**

```
'use client';

import { useState } from 'react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from '@/compone
import { Label } from '@/components/ui/label';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@/compone
import { Textarea } from '@/components/ui/textarea';
import { CopyIcon } from 'lucide-react'; // Assuming Lucide React i

export default function DashboardPage() {
    const [youtubeUrl, setYoutubeUrl] = useState('');
    const [isLoading, setIsLoading] = useState(false);
    const [generatedOutputs, setGeneratedOutputs] = useState({
        outputX: '',
        outputLinkedIn: '',
        outputTelegram: '',
    });

    const handleGenerate = async () => {
        setIsLoading(true);
        try {
            // ... (Credit deduction logic)

            // ... (Transcript fetching logic)

            // 3. AI Content Generation
            const generateResponse = await fetch('/api/generate-con
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ rawTranscript, videoUrl: you
            });

            if (!generateResponse.ok) {
                const errorData = await generateResponse.json();
                throw new Error(errorData.error || 'Failed to gener
            }

            const generatedContent = await generateResponse.json();
            setGeneratedOutputs(generatedContent);
```

```
        } catch (error) {
            console.error('Generation error:', error);
            alert(`Error: ${error.message}`);
        } finally {
            setIsLoading(false);
        }
    };

    const copyToClipboard = (text: string) => {
        navigator.clipboard.writeText(text);
        alert('Copied to clipboard!');
    };

    return (
        <div className="min-h-screen bg-gray-900 text-white p-8">
            <div className="max-w-3xl mx-auto">
                <h1 className="text-4xl font-bold mb-8 text-center"
                <Card className="bg-gray-800 border-gray-700 mb-8">
                    <CardHeader>
                        <CardTitle className="text-white">Generate
                    </CardHeader>
                    <CardContent>
                        <div className="space-y-4">
                            <div>
                                <Label htmlFor="youtube-url" classN
                                <Input
                                    id="youtube-url"
                                    type="url"
                                    placeholder="e.g., https://www.
                                    value={youtubeUrl}
                                    onChange={(e) => setYoutubeUrl(
                                    className="bg-gray-700 border-g
                                />
                            </div>
                            <Button
                                onClick={handleGenerate}
                                disabled={!youtubeUrl || isLoading}
                                className="w-full bg-blue-600 hover
                            >
                                {isLoading ? 'Generating...' : 'Gen
                            </Button>
                        </div>
                    </CardContent>
                </Card>
```

```
                {generatedOutputs.outputX && (
                    <Card className="bg-gray-800 border-gray-700">
                        <CardHeader>
                            <CardTitle className="text-white">Gener
                        </CardHeader>
                        <CardContent>
                            <Tabs defaultValue="x">
                                <TabsList className="grid w-full gr
                                    <TabsTrigger value="x" classNam
                                    <TabsTrigger value="linkedin" c
                                    <TabsTrigger value="telegram" c
                                </TabsList>
                                <TabsContent value="x" className="m
                                    <div className="relative">
                                        <Textarea value={generatedO
                                        <Button size="sm" className
                                            <CopyIcon className="h-
                                        </Button>
                                    </div>
                                </TabsContent>
                                <TabsContent value="linkedin" class
                                    <div className="relative">
                                        <Textarea value={generatedO
                                        <Button size="sm" className
                                            <CopyIcon className="h-
                                        </Button>
                                    </div>
                                </TabsContent>
                                <TabsContent value="telegram" class
                                    <div className="relative">
                                        <Textarea value={generatedO
                                        <Button size="sm" className
                                            <CopyIcon className="h-
                                        </Button>
                                    </div>
                                </TabsContent>
                            </Tabs>
                        </CardContent>
                    </Card>
                )}
            </div>
        </div>
```

```
    );
}
```

3. **Install Lucide React (if not already):**

```
npm install lucide-react
```

4. **Install shadcn/ui Textarea component:**

```
npx shadcn-ui@latest add textarea
```

4.5.2. Acceptance Criteria for Phase 5:

- Generated content for Twitter/X, LinkedIn, and Telegram is displayed in a tabbed interface on the dashboard.
- Each tab shows the respective generated content in a read-only textarea.
- A “Copy to Clipboard” button is present for each content output and successfully copies the text.

Phase 6: Paywall & Pricing Component

Objective: Implement a paywall mechanism that redirects users with zero credits to a pricing/payment page.

4.6.1. Redirect to Pricing Page

- **Requirements:** When `deduct_credit` returns insufficient credits, redirect the user to a dedicated pricing page.
- **Context:** Guides users towards monetization options.
- **Code Logic Instructions:**
  1. **Create Pricing Page (`src/app/pricing/page.tsx`):**

```tsx
import { Card, CardContent, CardHeader, CardTitle, CardDescription
import { Button } from '@/components/ui/button';

export default function PricingPage() {
    return (
        <div className="min-h-screen bg-gray-900 text-white p-8 fle
            <div className="max-w-4xl mx-auto text-center">
                <h1 className="text-4xl font-bold mb-8">Upgrade You
                <p className="text-lg text-gray-300 mb-12">Unlock m

                <div className="grid grid-cols-1 md:grid-cols-3 gap
                    <Card className="bg-gray-800 border-gray-700 fl
                        <CardHeader>
                            <CardTitle className="text-white">Free<
                            <CardDescription className="text-gray-4
                        </CardHeader>
                        <CardContent className="flex-grow flex flex
                            <div className="text-center mb-6">
                                <span className="text-5xl font-bold
                                <span className="text-gray-400">/mo
                            </div>
                            <ul className="text-gray-300 text-left
                                <li>✓ 3 Generations/month</li>
                                <li>✓ Basic AI models</li>
                                <li>× Advanced features</li>
                            </ul>
                            <Button disabled className="w-full bg-g
                        </CardContent>
                    </Card>

                    <Card className="bg-gray-800 border-gray-700 fl
                        <CardHeader>
                            <CardTitle className="text-white">Pro</
                            <CardDescription className="text-gray-4
                        </CardHeader>
                        <CardContent className="flex-grow flex flex
                            <div className="text-center mb-6">
                                <span className="text-5xl font-bold
                                <span className="text-gray-400">/mo
                            </div>
                            <ul className="text-gray-300 text-left
                                <li>✓ 50 Generations/month</li>
```

```
                                <li>✓ Advanced AI models</li>
                                <li>✓ Priority support</li>
                            </ul>
                            <Button className="w-full bg-blue-600 h
                        </CardContent>
                    </Card>

                    <Card className="bg-gray-800 border-gray-700 fl
                        <CardHeader>
                            <CardTitle className="text-white">Premi
                            <CardDescription className="text-gray-4
                        </CardHeader>
                        <CardContent className="flex-grow flex flex
                            <div className="text-center mb-6">
                                <span className="text-5xl font-bold
                                <span className="text-gray-400">/mo
                            </div>
                            <ul className="text-gray-300 text-left
                                <li>✓ Unlimited Generations</li>
                                <li>✓ All Pro features</li>
                                <li>✓ Dedicated account manager</li
                            </ul>
                            <Button className="w-full bg-green-600
                        </CardContent>
                    </Card>
                </div>
            </div>
        </div>
    );
}
```

2. **Update `handleGenerate` in `src/app/dashboard/page.tsx` for redirection:**

```
// ... inside DashboardPage component
import { useRouter } from 'next/navigation';

export default function DashboardPage() {
    // ... existing states
    const router = useRouter();

    const handleGenerate = async () => {
        setIsLoading(true);
        try {
            // 1. Deduct Credit
            const deductResponse = await fetch('/api/deduct-credit'
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
            });

            if (!deductResponse.ok) {
                const errorData = await deductResponse.json();
                if (deductResponse.status === 402) {
                    alert('Insufficient credits. Redirecting to pri
                    router.push('/pricing'); // Redirect to pricing
                } else {
                    throw new Error(errorData.error || 'Failed to d
                }
                return; // Stop execution if credit deduction fails
            }
            const { newCredits } = await deductResponse.json();
            console.log('Credits remaining:', newCredits);
            // TODO: Update UI to reflect newCredits (e.g., a credi

            // ... (Transcript fetching logic)
            // ... (AI Content Generation logic)

        } catch (error) {
            console.error('Generation error:', error);
            alert(`Error: ${error.message}`);
        } finally {
            setIsLoading(false);
        }
    };
```

```
    // ... rest of the component
}
```

4.6.2. Acceptance Criteria for Phase 6:

- A dedicated `/pricing` page is created with placeholder pricing plans.
- When a user attempts to generate content with 0 credits, they are redirected to the `/pricing` page.
- The redirection is handled client-side using `next/navigation`.

Phase 7: User Credit Display & History

Objective: Display the user's current credit count and a history of their past generations.

4.7.1. Display Current Credits (Updated)

- **Requirements:** Fetch and display the user's current credit count on the dashboard using the new `@supabase/ssr` server client.
- **Context:** Provides immediate feedback to the user about their usage.
- **Code Logic Instructions:**
  1. **API Route to Get User Credits (`src/app/api/user-credits/route.ts`):**

```
import { NextResponse } from 'next/server';
import { createServerSupabaseClient } from '@/lib/supabase/server';

export async function GET(request: Request) {
    const supabase = createServerSupabaseClient(); // Updated clien
    const { data: { user } } = await supabase.auth.getUser();

    if (!user) {
        return NextResponse.json({ error: 'Unauthorized' }, { statu
    }

    try {
        const { data, error } = await supabase
            .from('users')
            .select('credits')
            .eq('id', user.id)
            .single();

        if (error) {
            console.error('Supabase fetch credits error:', error);
            return NextResponse.json({ error: error.message }, { st
        }

        return NextResponse.json({ credits: data.credits });
    } catch (error) {
        console.error('Error fetching user credits:', error);
        return NextResponse.json({ error: 'Internal server error' }
    }
}
```

2. **Update Dashboard to Fetch and Display Credits:**

```
'use client';

import { useState, useEffect } from 'react';
// ... other imports

export default function DashboardPage() {
    // ... existing states
    const [userCredits, setUserCredits] = useState<number | null>(n

    useEffect(() => {
        const fetchCredits = async () => {
            const response = await fetch('/api/user-credits');
            if (response.ok) {
                const data = await response.json();
                setUserCredits(data.credits);
            }
        };
        fetchCredits();
    }, []);

    // ... (handleGenerate function, update newCredits state after

    return (
        <div className="min-h-screen bg-gray-900 text-white p-8">
            <div className="max-w-3xl mx-auto">
                {/* ... Dashboard Title and Credits Display */}

                {userCredits !== null && (
                    <p className="text-center text-lg mb-4">Credits
                )}
                {/* ... rest of the dashboard UI */}
            </div>
        </div>
    );
}
```

4.7.2. Display Generation History (Updated)

- **Requirements:** Fetch and display a list of past generations from the `generations` table using the new `@supabase/ssr` server client.

- **Context:** Allows users to review and re-access their previously generated content.
- **Code Logic Instructions:**
    1. **API Route to Get Generation History (`src/app/api/generation-history/route.ts`):**

```
import { NextResponse } from 'next/server';
import { createServerSupabaseClient } from '@/lib/supabase/server';

export async function GET(request: Request) {
    const supabase = createServerSupabaseClient(); // Updated clien
    const { data: { user } } = await supabase.auth.getUser();

    if (!user) {
        return NextResponse.json({ error: 'Unauthorized' }, { statu
    }

    try {
        const { data, error } = await supabase
            .from('generations')
            .select('*')
            .eq('user_id', user.id)
            .order('created_at', { ascending: false });

        if (error) {
            console.error('Supabase fetch generations error:', erro
            return NextResponse.json({ error: error.message }, { st
        }

        return NextResponse.json({ history: data });
    } catch (error) {
        console.error('Error fetching generation history:', error);
        return NextResponse.json({ error: 'Internal server error' }
    }
}
```

    2. **Update Dashboard to Display History:**

```tsx
'use client';

import { useState, useEffect } from 'react';
// ... other imports
import { Table, TableBody, TableCell, TableHead, TableHeader, Table

interface GenerationRecord {
    id: string;
    video_url: string;
    created_at: string;
    output_x: string;
    output_linkedin: string;
    output_telegram: string;
}

export default function DashboardPage() {
    // ... existing states
    const [generationHistory, setGenerationHistory] = useState<Gene

    useEffect(() => {
        // ... fetchCredits logic

        const fetchHistory = async () => {
            const response = await fetch('/api/generation-history')
            if (response.ok) {
                const data = await response.json();
                setGenerationHistory(data.history);
            }
        };
        fetchHistory();
    }, []);

    // ... (handleGenerate function)

    return (
        <div className="min-h-screen bg-gray-900 text-white p-8">
            <div className="max-w-3xl mx-auto">
                {/* ... Dashboard Title and Credits Display */}

                {/* ... Generate Content Card */}

                {/* ... Generated Content Display */}
```

```
                <Card className="bg-gray-800 border-gray-700 mt-8">
                    <CardHeader>
                        <CardTitle className="text-white">Generatio
                    </CardHeader>
                    <CardContent>
                        {generationHistory.length > 0 ? (
                            <Table>
                                <TableHeader>
                                    <TableRow>
                                        <TableHead className="text-
                                        <TableHead className="text-
                                        <TableHead className="text-
                                    </TableRow>
                                </TableHeader>
                                <TableBody>
                                    {generationHistory.map((record)
                                        <TableRow key={record.id} c
                                            <TableCell className="t
                                            <TableCell className="t
                                            <TableCell>
                                                <Button variant="ou
                                                    View Outputs
                                                </Button>
                                            </TableCell>
                                        </TableRow>
                                    ))}
                                </TableBody>
                            </Table>
                        ) : (
                            <p className="text-gray-400 text-center
                        )}
                    </CardContent>
                </Card>
            </div>
        </div>
    );
}
```

3. **Install shadcn/ui Table component:**

```
npx shadcn-ui@latest add table
```

4.7.3. Acceptance Criteria for Phase 7:

- The user's current credit count is displayed prominently on the dashboard.
- A list of past generations is fetched and displayed in a table format.
- Each history entry shows the video URL and generation timestamp.
- Clicking an action button on a history entry populates the generated content display with the respective outputs.

Phase 8: Deployment & Monitoring

Objective: Deploy the application to Vercel and set up basic monitoring.

4.8.1. Vercel Deployment

- **Requirements:** Deploy the Next.js application to Vercel.
- **Context:** Makes the application publicly accessible.
- **Code Logic Instructions:**
  1. **Vercel CLI:**

```
npm install -g vercel
vercel login
vercel
# Follow prompts to link your project and deploy.
```

  2. **Environment Variables on Vercel:** Ensure `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, and `ANTHROPIC_API_KEY` are configured as environment variables in your Vercel project settings.

4.8.2. Basic Monitoring

- **Requirements:** Utilize Vercel's built-in analytics and logging for basic monitoring.
- **Context:** Provides insights into application performance and errors.

- **Code Logic Instructions:**
    1. **Vercel Dashboard:** Access your project dashboard on Vercel to view logs, analytics, and performance metrics.
    2. **Supabase Dashboard:** Monitor database performance, RLS logs, and function invocations via the Supabase dashboard.

4.8.3. Acceptance Criteria for Phase 8:

- The application is successfully deployed to Vercel and accessible via a public URL.
- All environment variables are correctly configured on Vercel.
- Basic logs and analytics are available through the Vercel dashboard.
- Supabase monitoring tools can be used to observe backend operations.

5. Future Enhancements (Beyond MVP)

- **Advanced AI Models:** Integration with other LLMs (e.g., OpenAI GPT-4, Google Gemini) for varied content styles.
- **Audio File Upload:** Allow users to upload audio files directly for transcription via OpenAI Whisper API.
- **Custom Prompts:** Enable users to define and save their own custom prompts for content generation.
- **Scheduled Generations:** Allow scheduling content generation for future publication.
- **Direct Publishing:** Integration with Twitter/X, LinkedIn, and Telegram APIs for direct posting.
- **Stripe/Crypto Payment Gateway:** Full integration with a payment gateway for credit purchases and subscription management.
- **Admin Dashboard:** A backend interface for managing users, credits, and generations.
- **Analytics:** Detailed analytics on content performance and user engagement.

6. References

[1] Supabase Documentation: Creating a Supabase client for SSR. (n.d.). Retrieved from https://supabase.com/docs/guides/auth/server-side/creating-a-client [2] Supabase Documentation: Migrating to the SSR package from Auth Helpers. (n.d.). Retrieved from https://supabase.com/docs/guides/auth/server-side/migrating-to-ssr-from-auth-helpers [3] Supabase Documentation: Database Functions. (n.d.). Retrieved from

https://supabase.com/docs/guides/database/functions [4] DEV Community: Building a YouTube Transcript API with Next.js. (2026, February 2). Retrieved from https://dev.to/hammer_nexon_0e3907d1ade2/building-a-youtube-transcript-api-with-nextjs-1mh8 [5] Anthropic Documentation: Claude 3.5 Sonnet. (n.d.). Retrieved from https://www.anthropic.com/news/claude-3-5-sonnet [6] Vercel AI SDK Documentation. (n.d.). Retrieved from https://sdk.vercel.ai/docs [7] shadcn/ui Documentation. (n.d.). Retrieved from https://ui.shadcn.com/docs [8] Lucide React Documentation. (n.d.). Retrieved from https://lucide.dev/icons/ [9] Next.js Documentation. (n.d.). Retrieved from https://nextjs.org/docs [10] Tailwind CSS Documentation. (n.d.). Retrieved from https://tailwindcss.com/docs [11] Reddit: r/Supabase - Atomic transactions and rollback?. (2024, March 31). Retrieved from https://www.reddit.com/r/Supabase/comments/1bs1prb/atomic_transactions_and_rollback/ [12] YouTube: Write Viral LinkedIn & X Posts With Claude 3.5 Sonnet (Tutorial). (2024, July 16). Retrieved from https://www.youtube.com/watch?v=kVyPVXSgPpA [13] YouTube: Generating YouTube Transcript via API Route Handlers – Part 6.2. (2024, April 18). Retrieved from https://www.youtube.com/watch?v=znZs418fc_c [14] Marmelab: Transactions and RLS in Supabase Edge Functions. (2025, December 8). Retrieved from https://marmelab.com/blog/2025/12/08/supabase-edge-function-transaction-rls.html